Noah Dataset Update v2 — Multi-Step Planning Support

> **Document Version:** 2.0
> **Status:** Active
> **Applies To:** All Noah datasets (Wallet, Product, Merchant, Offers, Navigation, Recommendation, etc.)

🎯 Objective

The first version of the Noah datasets successfully supports **single-step user requests**, where one user instruction maps to a single planner action.

This update extends the dataset specification to support **multi-step planning**, allowing Noah to understand and execute multiple actions from a single natural language request.

This update is **fully backward compatible** with all existing datasets.

💡 Why This Update?

Real users rarely ask only one thing.

Instead of saying:

> Find frozen pizzas below €2.

Users naturally ask:

> Find frozen pizzas below €2 and navigate me there.

or

> Show my Payback card and tell me my current points.

or

> I'm at Netto. What are today's ice cream offers below €3? Then display my Payback card because I'll pay using it.

These are **compound requests** that require Noah to execute multiple planner actions in sequence.

Rather than creating hundreds of combined intents such as:

- SEARCH_AND_NAVIGATE
- SEARCH_AND_PAY
- SEARCH_AND_COMPARE
- SEARCH_AND_OPEN_CARD

we allow Noah to generate an **ordered execution plan**.

✅ Existing Dataset

The following columns remain **unchanged**.

Existing Column
instruction
domain
intent
sub_intent
tool
response_mode
entity_product
entity_merchant
entity_brand
entity_category
entity_price_min
entity_price_max
entity_location
entity_radius
requires_memory
requires_rag
requires_recommendation
planner_action

No existing rows need to be modified.

➕ New Columns

Only four new columns are introduced.

Column	Purpose	Example
workflow_type	Defines execution style	single / sequential / parallel
planner_actions	Ordered planner action list	SEARCH_OFFERS > FILTER_PRICE > DISPLAY_CARD
planner_action_count	Number of planner actions	3
tool_sequence	Ordered backend tool execution	offer_engine > offer_engine > wallet_api

---

📌 Column Details

1. workflow_type

Defines how the planner should execute the request.

Possible values:

- single
- sequential
- parallel

Example

User:

> Show my Payback card.

workflow_type

> single

User:

> Find frozen pizzas and navigate me there.

workflow_type

> sequential

Future example

> Compare prices at Lidl and REWE simultaneously.

workflow_type

> parallel

---

2. planner_actions

The current datasets contain only one planner action.

Instead, this field stores the **entire execution sequence**.

Old

> SEARCH_PRODUCT

New

> SEARCH_PRODUCT → NAVIGATE

Another example

> SEARCH_OFFERS → FILTER_PRICE → DISPLAY_CARD

WATERMARK WATERMARK WATERMARK

3. planner_action_count

Stores the total number of planner actions.

Examples

- 1
- 2
- 3
- 4

This field is mainly useful for:

- debugging
- statistics
- evaluation
- model benchmarking

---

4. tool_sequence

Maps each planner action to the backend component responsible for execution.

Example

Planner Actions

> SEARCH_PRODUCT → NAVIGATE

Tool Sequence

> merchant_engine → google_maps

Another example

Planner Actions

> DISPLAY_CARD → FETCH_POINTS

Tool Sequence

> wallet_api → loyalty_api

Keeping planner actions independent from backend tools makes Noah much easier to extend in the future.

📊 Updated Dataset Example

instruction	planner_action	workflow_type	planner_actions	planner_action_count	tool_sequence
Find frozen pizzas below €2	SEARCH_PRODUCT	single	SEARCH_PRODUCT	1	merchant_engine

instruction	planner_action	workflow_type	planner_actions	planner_action_count	tool_sequence
Find frozen pizzas below €2 then navigate there	SEARCH_PRODUCT	sequential	SEARCH_PRODUCT → NAVIGATE	2	merchant_engine → google_maps
Show my Payback card then display my points	DISPLAY_CARD	sequential	DISPLAY_CARD → FETCH_POINTS	2	wallet_api → loyalty_api

Notice that **planner_action remains unchanged**, ensuring full compatibility with Version 1 datasets.

✍ Dataset Generation Guidelines

Each contributor should generate approximately **30–70 additional multi-step examples** for every dataset they own.

These examples should **extend** the dataset—not replace existing rows.

Every new example should:

- combine multiple existing planner actions
- preserve the current intent hierarchy
- use realistic user language
- avoid introducing unnecessary new intents
- maintain consistent entity labels
- follow logical execution order

✅ Good Multi-Step Examples

Wallet

> Show my Payback card and then display my current points.

Product Search

> Find frozen pizzas below €2 and navigate me there.

Merchant

> Find the closest REWE and open its store page.

Offers

> Show today's Lidl offers and add the cheapest milk to my shopping list.

Recommendation

> Recommend a healthy cereal below €5 and compare it with another option.

Shopping List

> Add eggs, milk and bread to my shopping list and estimate the total cost.

Navigation

> Navigate me to the nearest Kaufland and display today's opening hours.

📝 Annotation Rules

When generating paraphrases:

✔ Keep all entities identical.

✔ Preserve planner actions.

✔ Preserve execution order.

✔ Only rewrite the natural language.

Example

Canonical prompt

> Find frozen pizzas below €2 then navigate me there.

Possible paraphrases

- Show me frozen pizzas under €2 and guide me there.
- Find cheap frozen pizzas nearby and start navigation.
- Locate frozen pizzas below €2 and take me there.
- I need frozen pizzas under €2. Open directions afterwards.

All of these should produce **exactly the same planner actions**.

💡 Worked Example

User Prompt

> I'm at Netto. What are the current offers on ice creams here? Fetch me something below €3 and then display my Payback card as I'll pay using it.

Workflow

WATERMARK WATERMARK WATERMARK

Planner Actions

Step	Planner Action
1	SEARCH_OFFERS
2	FILTER_PRICE
3	DISPLAY_CARD

Tool Sequence

Step	Backend Tool
1	offer_engine
2	offer_engine
3	wallet_api

Extracted Entities

Entity	Value
Merchant	Netto
Product	Ice Cream
Category	Frozen Desserts
Price Max	€3
Location	Current Location

🤖 Expected JSON Output


```
{
  "domain": "SHOPPING",
  "intent": "MULTI_STEP_TASK",
  "sub_intent": "OFFERS_AND_PAYMENT",

  "entities": {
    "merchant": "Netto",
    "product": "Ice Cream",
    "brand": null,
```

```
    "category": "Frozen Desserts",
    "price_min": null,
    "price_max": 3.0,
    "location": "CURRENT_LOCATION",
    "radius": null
  },

  "workflow_type": "sequential",

  "planner_action": "SEARCH_OFFERS",

  "planner_actions": [
    "SEARCH_OFFERS",
    "FILTER_PRICE",
    "DISPLAY_CARD"
  ],

  "planner_action_count": 3,

  "tool_sequence": [
    "offer_engine",
    "offer_engine",
    "wallet_api"
  ],

  "tool": "offer_engine",

  "response_mode": "hybrid",

  "requires_memory": false,

  "requires_rag": false,

  "requires_recommendation": false,

  "confidence": 0.99
}
```

✅ Contributor Checklist

For every new multi-step example:

- ☐ Uses an existing domain
- ☐ Uses existing intents wherever possible
- ☐ Adds 2–4 planner actions
- ☐ Preserves logical execution order
- ☐ Labels all entities correctly
- ☐ Keeps planner_action equal to the first planner step
- ☐ Fills workflow_type
- ☐ Fills planner_actions
- ☐ Fills planner_action_count
- ☐ Fills tool_sequence

🚀 Final Goal

Version 2 transforms Noah from a **single-intent classifier** into an **AI Planner** capable of generating executable workflows while remaining fully compatible with all existing datasets.

Instead of predicting only one backend action, Noah now learns to generate a complete execution plan that the Python backend can execute step-by-step, enabling complex conversational tasks without requiring a complete redesign of the existing datasets.

WATERMARK